

Nome do estudante: Adalbetus

Código: INPH

1.[8.0 pontos = 1.0 + 1.5 + 2.0 + 2.5 + 1.0]

a) [1.0] A função **toUpperStr()** converte todas as letras da *string*, *s*, que recebe como parâmetro, para maiúsculas. O resultado da conversão deve ser guardado na mesma *string*. Complete o protótipo e escreva o código desta função.

```
void toUpperStr(string* s)
{
    for (int i = 0; i < s->length(); i++)
    {
        if (s->at(i) >= 'a' && s->at(i) <= 'z')
        {
            s->at(i) += ('A' - 'a');
        }
    }
}
```

b) [1.5] A função **transformLine()** recebe como parâmetro uma *string*, *s*, constituída por letras, espaços e outros caracteres e retorna uma *string* constituída apenas por letras maiúsculas e espaços, substituindo os caracteres que não são letras nem espaços pelo carácter espaço, ' '. Complete o protótipo e escreva o código desta função. A *string* recebida como parâmetro não será modificada. **Nota:** para fazer a conversão para maiúsculas deve usar a função **toUpperStr()**, da alínea anterior.

Exemplo: se a *string* *s* for "A very, very short sentence!" a *string* retornada deverá ser "A VERY VERY SHORT SENTENCE".

```
string transformLine(string s)
{
    string returnString = "";
    for (int i = 0; i < s.length(); i++)
    {
        if (!(s.at(i) >= 'a' && s.at(i) <= 'z') &&
            !(s.at(i) >= 'A' && s.at(i) <= 'Z') && s.at(i) != ' ')
        {
            returnString += ' ';
        }
        else
        {
            returnString += s.at(i);
        }
    }
    toUpperStr(&returnString);
    return returnString;
}
```

c) [2.0] A função **decomposeLine()** recebe como parâmetro uma linha de texto, *line* (uma *string*), e devolve através de outro parâmetro, *words* (um **vector<string>**), as palavras constituintes dessa linha de texto. Esta função deve começar por invocar **transformLine()** para substituir por espaços todos os caracteres da linha de texto que não sejam letras nem espaços. Complete o protótipo e escreva o código daquela função. **Nota:** pode haver mais do que um espaço entre palavras (ver exemplo da alínea anterior). **Sugestão:** recorra a uma *stringstream* para fazer a decomposição.

Exemplo: o vetor resultante da decomposição da *string* do exemplo será{"A", "VERY", "VERY", "SHORT", "SENTENCE"}.

```
void decomposeLine(string line, vector<string>* words)
{
    line = transformLine(line);
    line.erase(line.find_last_not_of(' ')+1, line.length());
    while (line.length() > 0)
    {
        line.erase(0, line.find_first_not_of(' '));
        int pos = line.find_first_of(' ');
        string sub = line.substr(0, pos);
        if (sub.find(" ") == sub.npos)
        {
            words->push_back(sub);
        }
        line.erase(0, pos);
    }
}
```

d) [2.5] Escreva o código de um programa que, usando a função **decomposeLine()**, gera uma lista com todas as palavras contidas no ficheiro "**text.txt**", guardado na pasta "**C:\docs**", e grava as palavras resultantes no ficheiro "**words.txt**", na pasta atual. As palavras resultantes devem ser escritas por ordem alfabética, uma palavra por cada linha do ficheiro.

Notas: **1)** omita os ficheiros de inclusão; **2)** considere que é sempre possível abrir/ler/escrever os ficheiros com sucesso; **3)** se houver palavras repetidas, elas deverão ser mantidas na lista de palavras resultantes; **4)** não repita o código das funções descritas nas alíneas anteriores; indique apenas onde deveriam ser colocadas, no programa.

```
ifstream inputFile("C:\\docs\\text.txt");
vector<string> total;
string line;
while (getline(inputFile, line))
{
    vector<string> v;
    decomposeLine(line, &v);
    total.insert(total.end(), v.begin(), v.end());
}
inputFile.close();
sort(total.begin(), total.end());
ofstream outFile("C:\\docs\\words.txt");
for (string s : total)
{
    outFile << s << endl;
}
outFile.close();
```

e) [1.0] Indique a(s) alteração(ões) a introduzir no programa da alínea anterior de modo que, na lista de palavras, não surjam palavras repetidas.

```
/*Passar o conteudo para um set*/
set<string> strSet;
for (string s : total)
{
    strSet.insert(s);
}
total = vector<string>();
for (string s : strSet)
{
    total.push_back(s);
}
```

Nome do estudante: _____ Código: _____

2.[8.0 pontos = 1.0 + 1.5 + 1.5 + 2.0 + 2.0]

O jogo da Batalha Naval é jogado num tabuleiro bidimensional. O jogo consiste em adivinhar as posições em que o jogador adversário colocou a sua "armada", constituída por vários "navios", cada um representado no tabuleiro por um conjunto de células consecutivas, dispostas linearmente, na direção vertical ou na horizontal. Apresenta-se a seguir a declaração parcial de algumas estruturas de dados usadas para implementar esse jogo e uma ilustração da representação interna de um tabuleiro, no qual estão colocados 5 "navios", identificados pelos números 1 a 5; o "mar" é representado pelo valor -1.

```

Struct
    Position{ int
        lin, col;
    };

// =====

class Ship{
public:
    Ship(unsigned int identifier, char symbol,
        Position position, char direction,
        size_t size);
    unsigned int id() const; //returns identifier
    Position pos() const; //returns
    positionchar dir() const; //returns
    directionsize_t size() const; //returns
    size
    // ... OTHER METHODS
private:
    unsigned int identifier; // ship id number
    // ... OTHER ATTRIBUTES AND/OR METHODS
};

```

```

class Board{
public:
    Board(size_t numLines = 10, size_t numColumns = 10);
    bool putShip(const Ship &s); //add ship to board, if possible
    // ... OTHER METHODS
private:
    bool canPutShip(Position pos, char dir, size_t size);
    size_t numLines, numColumns;
    vector<vector<int>> board; // each element = ship id or -1
    vector<Ship> ships;
    // ... OTHER ATTRIBUTES AND/OR METHODS
};

```

Possível conteúdo		-1 3 -1 -1 -1 -1 -1 -1 -1 -1
de vector<vector<int>>		-1 3 -1 -1 -1 -1 -1 5 5 -1
board com 7x10 células	→	-1 3 -1 1 1 -1 -1 -1 -1 -1
		-1 3 -1 -1 -1 2 -1 -1 -1 -1
		-1 3 -1 -1 -1 2 -1 4 4 4
-1 = "mar"		-1 -1 -1 -1 -1 2 -1 -1 -1 -1
1,2,...5 = "navio" 1,...,navio" 5		-1 -1 -1 -1 -1 2 -1 -1 -1 -1

a) [1.0] Explique por que é possível fazer a declaração **Board b**; mas não a declaração **Ship s**;

Não há um construtor padrão para a classe Ship.
Já a o construtor da classe Board possui valores padrão para seus parâmetros.

b) [1.5] Implemente o construtor da classe **Board**, o qual deve preencher todos os elementos do atributo **board** com o valor -1 que indica que a célula respetiva corresponde a "mar livre" (isto é, sem "navios").

```

Board::Board(size_t numLines, size_t numColumns)
{
    for (int i = 0; i < numLines; i++)
    {
        vector<int> v;
        for (int j = 0; j < numColumns; j++)
        {
            v.push_back(-1);
        }
        board.push_back(v);
    }
}

```

c) [1.5] O método **canPutShip()**, da classe **Board**, determina se o "navio" pode ser colocado na posição **pos**(posição do canto superior esquerdo do "navio") e na direção **dir**. Para isso, é necessário verificar que não ultrapassa os limites do tabuleiro e que todas as células que vai ocupar estão livres (valor **-1**). Complete o código abaixo apresentado, nas partes assinaladas. **Nota**: a direção do "navio" é indicada por uma das letras maiúsculas seguintes: **H**(=horizontal) ou **V**(=vertical).

```
bool Board::canPutShip(Position pos, char dir, size_t size) {
    switch (dir)
    {
        case 'H': // TO DO: test if ship with length 'size' can be put at 'pos', in horizontal
direction
        {
            if (pos.col + size - 1 >= board.at(0).size() || pos.lin >= board.size())
            {
                return false;
            }
            else
            {
                for (int i = 0; i < size; i++)
                {
                    if (board.at(pos.lin).at(pos.col + i) != -1)
                    {
                        return false;
                    }
                }
                break;
            }

            case 'V': // TO DO, just this line: test if ship ... can be put in vertical direction
// DONE. This piece of code was correctly implemented but was carelessly
erased...
        }
        return true;
    }
}
```

d) [2.0] O método **putShip()** tenta colocar no tabuleiro (modificando os atributos **board** e **ships**) o "navio", **s**, que recebe como parâmetro. A posição onde se pretende colocar o navio é a que for retornada pelo método **pos()** de **Ship**. Escreva o código deste método, o qual retorna **true** ou **false**, consoante tenha sido possível ou não colocar o "navio" no tabuleiro. **Nota:** use o método **canPutShip()**, mesmo que não o tenha implementado, para verificar se o "navio" pode ou não ser colocado no ~~tabuleiro~~ **tabuleiro**.

```
bool Board::putShip(const Ship &s)
{
    if (canPutShip(s.pos(),s.dir(),s.size()))
    {
        switch (s.dir())
        {
            case 'H':
                for (int i = 0; i < s.size(); i++)
                {
                    board.at(s.pos().lin).at(s.pos().col + i) = s.identifier;
                }
                break;
            case 'V':
                for (int i = 0; i < s.size(); i++)
                {
                    board.at(s.pos().lin + i).at(s.pos().col) = s.identifier;
                }
                break;
        }
        return true;
    }
    return false;
}
```

e) [2.0] Escreva um pedaço de código que cria um tabuleiro com 10x20 células, lê do teclado os dados de um "navio"(identificador, símbolo, posição, direção e tamanho) e tenta colocar esse "navio" no tabuleiro. Se isso não for possível, lança um exceção, arremessando a *string* "Can't put ship!". **Nota:** considere que todos os valores lidos para os atributos do "navio" são válidos, isto é, do tipo correto.

```
Board b(10,20);
unsigned id;
char symbol, direction;
int lin, col, size;
cout << "ID"; cin >> id;
cout << "Symbol"; cin >> symbol;
cout << "Position line"; cin >> lin;
cout << "Position column"; cin >> col;
cout << "Direction"; cin >> direction;
cout << "Size"; cin >> size;
Ship s(id, symbol, Position{ lin,col }, direction, size);
if (!b.putShip(s))
{
    throw exception("Can't put ship!");
}
```

Nome do estudante: _____ Código: _____

3.[4.0 pontos = 1.0 + 1.0 + 1.0 + 1.0]

a)[1.0]

a1)A função

`float average(const int grades[], int numStudents)` faz parte de um programa que permite calcular a média das classificações obtidas pelos estudantes numa prova. Escreva o código da função `average()`.

Considere que `numStudents` é sempre um valor maior ou igual que 1.

a2)O número de estudantes que realiza cada prova é variável. Pretende-se que o programa use o espaço estritamente necessário para guardar as classificações. Escreva o pedaço de código que lê do teclado o número de estudantes e as suas classificações (após reservar o espaço necessário para estas), e mostra no ecrã a média das classificações.

Nota: considere que o programa tem uma função, já implementada, que lê as classificações:

`void readGrades(int grades[], int numStudents)`

```
float sum = 0;
for (int i = 0; i < numStudents; i++)
{
    sum += grades[i];
}
return sum / numStudents;
```

```
int numStudents;
int* g;
cout << "N: "; cin >> numStudents;
readGrades(g, numStudents);
cout << average(g, 4) << endl;
```

b) [1.0] A STL de C++ disponibiliza uma função `find()` cujo *template* é o seguinte:

```
template <class InputIterator, class T>
InputIterator find (InputIterator first, InputIterator last, const T& val);
```

em que o parâmetro `val` é o valor a procurar. Diga se é possível usar esta função para procurar um valor numa variável `v`, do tipo `vector<string>`. Em caso afirmativo escreva o pedaço de código que permite procurar o valor "31" em `v`, escrevendo uma mensagem adequada, que indique se o valor foi ou não encontrado. Justifique a sua resposta.

```
auto s = find(v.begin(), v.end(), "31");
if (s != v.end())
{
    cout << "Element found";
}
else
{
    cout << "Element not found";
}
//A função find retorna um iterador apontando para o valor procurado caso tenha sido
encontrado e para o fim do vetor onde foi feita a busca caso contrário
```

c) [1.0] Um programador queria usar num programa uma estrutura de dados do tipo `set<Position>`, em que `Position` é o tipo de dados declarado no código da pergunta 2, mas verificou que acontecia um erro de compilação quando declarava variáveis deste tipo de `set`. Indique uma possível causa deste problema e aponte uma solução. Não escreva código.

Não tenho certeza dessa resposta.

A struct internamente usa algum tipo de operador que não possui uma sobrecarga para a struct `Position`, poderia se criar uma sobrecarga válida sabendo-se qual o operador que gera o erro.

d) [1.0] Considere as classes `Base` e `Derived` bem como a função `main()` de um programa que as usa, abaixo apresentadas.

```
Class
Base{ public:
    Base(int a) { _a = a; }
    virtual void show() const { cout << _a; }
protected:
    int _a;
};
//=====
class Derived: public Base{ public:
    Derived(int a, int b) : Base(a) { _b = b; } void
    show() const { cout << _a << ' ' << _b; }
private:
    int _b;
};
//=====
int main()
{
    Derived d(2, 3); d.show();
    cout << endl; Base b = d;
    b.show(); cout << endl; Base
    *pb = &d; pb->show(); cout
    << endl; 7
}
```

Qual a saída do programa?

2 3
2
2 3

Qual seria a saída do programa se o qualificativo `virtual` fosse retirado da função `show()` da classe `Base`?

2 3
2
2

O que aconteceria se o qualificativo `protected` da classe `Base` fosse substituído por `private`? Justifique brevemente a resposta.

Erro de compilação, pois desta maneira o atributo `_a` se torna exclusivo da classe base, não sendo levado a classe derived, logo inacessível